10 — Detailed Solution Design

Task 10 — Sprint 3 — Foundation & Planning

The implementation contract. Another engineer (or agent) should be able to build the system from this document plus `13_Prompt_Engineering_Framework.md` and `14_FastAPI_Service_Setup_Plan.md`.

---

1. Design Overview

```
Request → Auth/RBAC → InputGuardrail → Router ─── C1 RAG subgraph
                                                  ?? C2 SQL subgraph
                                                  ?? C3 Hybrid subgraph (parallel fan-out)
                                                  ?? C4 HighRisk subgraph (3-way fan-out + deep eval)
                                                  ?? C5 Escalation (terminal)
  → EscalationCheck (E1–E8, deterministic) → Synthesis (tool-less) → OutputValidator → Response
```

Invariants (from `06` —8): tool-less synthesis — only Risk assigns risk level — templated read-only SQL — deterministic escalation after every evidence node — citation-or-refusal — state-only agent communication — span per node — disclaimer always.

---

2. Shared State (`app/graph/state.py`)

```
class DocEvidence(TypedDict):
    chunk_id: str
    doc_id: str            # "02_Risk_Management_Exposure_Policy"
    doc_title: str
    doc_category: str      # strategy | risk | regulatory | market | monitoring | framework
    section: str
    page: int
    chunk_text: str
    score: float

class SqlEvidence(TypedDict):
    query_id: str          # "risk_logs.by_score_threshold"
    tables: list[str]
    params: dict
    row_count: int
    rows: list[dict]       # capped at MAX_SQL_ROWS
    executed_at: str

class RiskEvidence(TypedDict):
    asset_name: str
    risk_type: str
    risk_score: int
    risk_level: Literal["Low", "Medium", "High"]
    escalation_flag: bool
```

```
    assessment_date: str
    breached_thresholds: list[dict]   # {name, policy_value, observed_value, source_chunk_id}

class MarketEvidence(TypedDict):
    indicator_name: str
    region: str
    indicator_value: float
    measurement_date: str
    trend_direction: str
    interpretation: str               # grounded in doc 04 methodology
    source_chunk_id: str | None

class Conflict(TypedDict):
    policy_claim: str
    policy_chunk_id: str
    data_observation: str
    data_query_id: str
    severity: Literal["info", "warning", "breach"]

class GraphState(TypedDict):
    # ?? request ??
    trace_id: str
    session_id: str | None
    query: str
    normalized_query: str
    user_role: Literal["analyst", "risk_manager", "compliance", "admin"]
    allowed_portfolio_ids: list[int]

    # ?? understanding ??
    category: Literal["C1_RAG", "C2_SQL", "C3_HYBRID", "C4_HIGH_RISK", "C5_ESCALATION"] | None
    path: list[str]                   # agent node names, in execution order
    entities: dict                    # see —3.1
    preliminary_risk_level: str | None
    router_confidence: float

    # ?? evidence bundle ??
    doc_evidence: list[DocEvidence]
    sql_evidence: list[SqlEvidence]
    risk_evidence: list[RiskEvidence]
    market_evidence: list[MarketEvidence]
    conflicts: list[Conflict]

    # ?? risk assessment (Risk agent only writer) ??
    risk_level: str | None
    risk_rationale: str | None
    breached_thresholds: list[dict]
    any_escalation_flag: bool

    # ?? safety ??
    guardrail_hits: list[str]         # attack_type ids
    escalation_required: bool
    escalation_triggers: list[str]    # ["E2", "E6"]
    escalation_reason: str | None
    recommended_reviewer: str | None

    # ?? output ??
    answer: str | None
    citations: list[dict]
```

```
    confidence: float | None
    validator_retries: int

    # ?? memory ??
    prior_turns: list[dict]            # last 5 (ADR-006)
    carried_entities: dict

    # ?? ops ??
    agents_used: list[str]
    node_timings_ms: dict[str, int]
    token_usage: dict                  # {agent: {prompt, completion}}
    cost_usd: float
    errors: list[dict]                 # {node, error_type, message, recovered}
```

Write discipline: each node writes only its declared keys. Parallel nodes write to disjoint lists; LangGraph reducers append rather than replace.

Node	Writes
InputGuardrail	`guardrail_hits`, `escalation_required`, `escalation_triggers`
Router	`category`, `path`, `entities`, `preliminary_risk_level`, `router_confidence`, `normalized_query`
Retrieval	`doc_evidence`
PortfolioEval	`sql_evidence`
MarketData	`market_evidence`, `sql_evidence`
Risk	`risk_evidence`, `risk_level`, `risk_rationale`, `breached_thresholds`, `any_escalation_flag`, `conflicts`
EscalationCheck	`escalation_required`, `escalation_triggers`, `escalation_reason`, `recommended_reviewer`
Synthesis	`answer`, `citations`, `confidence`
Escalation	`answer` (handoff text), `citations`, `recommended_reviewer`
OutputValidator	`answer` (on repair/refusal), `validator_retries`
all	`agents_used`, `node_timings_ms`, `token_usage`, `cost_usd`, `errors`

---

3. Agent Contracts

3.1 Router Agent

Model	`LLM_MODEL_ROUTER`, temp 0
Input	`query`, `prior_turns`, `carried_entities`, `user_role`
Output	`category`, `path`, `entities`, `preliminary_risk_level`, `router_confidence`
Tools	none
Budget	<=350ms p95

Entity schema:

```
entities = {
  "portfolio_names": [str], "portfolio_ids": [int],
  "asset_names": [str], "asset_types": [str],      # Equity | ETF | Bond | Commodity
  "sectors": [str],                                 # Technology | Finance | Energy | Healthcare |
Consumer
  "regions": [str],                                 # US | Europe | Asia | Global (+ EU/APAC in
starter data)
  "indicators": [str],                              # Inflation Rate | Interest Rate | GDP Growth
| ...
  "risk_types": [str],                              # Market Volatility | Liquidity | Regulatory |
Credit | Geopolitical
  "risk_levels": [str], "risk_score_threshold": int | None,
  "allocation_threshold": float | None,
  "approval_statuses": [str],                       # Approved | Pending | Rejected | Under Review
  "decision_types": [str],                          # BUY | SELL | HOLD | Reduce Exposure
  "date_range": {"from": str|None, "to": str|None},
  "policy_topics": [str],                           # diversification | exposure limits |
liquidity coverage | ...
  "action_intent": bool                             # does the user want to DO something, or just
know?
}
```

Classification procedure (deterministic pre-filter first, then LLM): 1. **Pre-filter (code):** if any override/bypass pattern matches → `C5_ESCALATION`, skip the LLM. 2. **Pre-filter (code):** if the query contains only schema/column tokens and no policy language (`risk_score`, `allocation_percentage`, `escalation_flag`, "list", "count", "show") → strong C2 prior. 3. **LLM classification** with the router prompt (`13` —2) returning strict JSON. 4. **Post-checks (code, can only escalate category, never de-escalate):** - policy language **and** data language present → `C3_HYBRID` - `action_intent=true` **and** (`risk_level=High` or High-risk framing "despite", "even though", "increase allocation to", "approve") → `C4_HIGH_RISK` - `router_confidence < 0.6` → treat as the **more expensive** of the candidate categories

Path assignment: | Category | Path | |---|---| | C1_RAG | `["retrieval", "synthesis"]` | | C2_SQL | `["portfolio_eval", "synthesis"]` | | C3_HYBRID | `[["retrieval", "portfolio_eval"], "risk", "synthesis"]` | | C4_HIGH_RISK | `[["retrieval", "portfolio_eval", "market_data"], "risk", "portfolio_eval_deep", "synthesis"]` | | C5_ESCALATION | `["escalation"]` |

3.2 Retrieval Agent

Model	`LLM_MODEL_AGENT` for rerank only
Input	`normalized_query`, `entities.policy_topics`, `category`
Output	`doc_evidence[]` (<=`TOP_K`)
Tools	`vector_search`, `keyword_search`
Budget	<=700ms p95 (400ms without rerank)

Procedure: build the retrieval query (original + expanded policy terms from `policy_topics`) ? dense HNSW cosine top-`FETCH_K` (20) → Postgres FTS `ts_rank` top-`FETCH_K` → weighted fusion (`1-HYBRID_KEYWORD_WEIGHT` dense, `HYBRID_KEYWORD_WEIGHT` sparse, normalized) → optional `doc_category` filter when the Router names a scoped intent → LLM rerank of the 20 candidates to `TOP_K=5` → drop anything below `RETRIEVAL_MIN_SCORE`.

Document routing hints: | Query topic | Prefer `doc_id` | |---|---| | diversification, allocation, rebalancing, risk-adjusted return | `01` | | exposure limits, market/credit/liquidity risk definitions, stress testing | `02` | | compliance, disclosure, governance controls, escalation procedure | `03` | | market trends, economic indicator interpretation, macro impact | `04` | | performance metrics, benchmark comparison, volatility, drawdown, attribution | `05` | | Basel III, capital adequacy, RWA, liquidity coverage | `06` | | risk scoring methodology, classification tiers, approval thresholds, escalation triggers | `07` |

Failure: empty result or all below threshold → write empty `doc_evidence` and let the escalation engine fire E5. Retrieval never fabricates a chunk.

3.3 Portfolio Evaluation Agent

Model	`LLM_MODEL_AGENT` for template selection + slot filling
Input	`entities`, `category`, `allowed_portfolio_ids`
Output	`sql_evidence[]`
Tools	SQL template registry (§5)
Budget	<=600ms p95 (LLM selection ~300ms + execution ~50ms)

Procedure: the LLM is given the template catalogue (ids, descriptions, slot schemas) and returns `[{query_id, params}]` (1–3 templates max). Code validates each `query_id` exists and each slot passes its validator; invalid → one repair attempt with the validation error, then give up and report "no supported query". Executor runs each, then the agent computes derived values in **Python, not the LLM**: sector concentration (`SUM(allocation_percentage) GROUP BY sector`), top-N holdings by `market_value`, allocation vs. a supplied threshold, count of holdings above a threshold, benchmark name lookup. Deep mode (C4) additionally computes implied post-change concentration when the query quantifies a change.

3.4 Market Data Agent

Input	`entities.indicators`, `entities.regions`, `entities.date_range`
Output	`market_evidence[]` (+ the underlying `sql_evidence`)
Tools	`market_indicators.*` templates
Budget	<=500ms p95

Procedure: fetch matching indicator rows (latest per indicator+region unless a date range is given) → interpret **only** using methodology retrieved from doc 04 (`source_chunk_id` recorded); if no methodology chunk is available, report the raw value and `trend_direction` with no interpretation. Missing indicator/region → state the gap in `market_evidence` as an explicit absence; never substitute model world knowledge about macroeconomics.

3.5 Risk Agent

Model	`LLM_MODEL_AGENT` for rationale only
Input	`sql_evidence`, `doc_evidence`, `market_evidence`, `entities`

Output	`risk_evidence[]`, `risk_level`, `risk_rationale`, `breached_thresholds[]`, `any_escalation_flag`, `conflicts[]`
Tools	`risk_logs.*` templates; deterministic scorer
Budget	<=900ms p95

Deterministic classification (code, authoritative — the LLM cannot override it):

```
def classify(score: int) -> str:
    if score < 30:  return "Low"
    if score < 70:  return "Medium"
    return "High"

# roll-ups
any_escalation_flag = any(r["escalation_flag"] for r in rows)   # DB truth
extreme             = any(r["risk_score"] > RISK_SCORE_ESCALATION for r in rows)   # >80
portfolio_risk_level = max(classify(r["risk_score"]) for r in rows)  # ordered Low<Medium<High
```

Threshold reconciliation: for each numeric limit extracted from `doc_evidence` (e.g. "max 10% single-asset allocation", "risk score above 70 requires committee approval"), compare against the observed value from `sql_evidence`. Every breach becomes a `breached_thresholds` entry recording `policy_value`, `observed_value`, and `source_chunk_id`. Policy limits are parsed with a numeric extractor and **confirmed by the LLM against the chunk text** — a limit the LLM cannot point at in the chunk is discarded, not used.

Conflict detection: policy claim vs. data observation mismatch → `Conflict` with severity `breach` (limit exceeded), `warning` (approaching, within 10%), or `info` (definitional difference).

Staleness rule: `assessment_date` older than 180 days on a High-risk asset → flag as stale (feeds E3).

3.6 Synthesis Agent

Model	`LLM_MODEL_AGENT`, temp 0
Input	the full evidence bundle + `category` + `risk_*`
Output	`answer`, `citations[]`, `confidence`
Tools	**none** (ADR-008)
Budget	<=800ms p95

Rules: assert only what the bundle supports — attach a citation to every claim, mandatory on any sentence containing a number/percentage/threshold/asset name — for C3/C4 include >=1 `DOC:` and >=1 `SQL:` citation — state conflicts explicitly with both citations — never recommend an action (state what policy requires and what the data shows) — never mention internal component names — always end with the disclaimer. Output structure by category is specified in `13` —7.

`confidence` is computed, not vibed: `0.4`—`mean(top-3 retrieval scores) + 0.3`—`(evidence completeness for the category) + 0.3`—`router_confidence`, capped at 0.5 when any conflict has severity `breach`.

3.7 Escalation Agent

Model	`LLM_MODEL_AGENT` for the summary text only
Input	`escalation_triggers`, evidence bundle, `query`, `user_role`
Output	handoff envelope in `answer` + `citations` + `recommended_reviewer`
Tools	none
Budget	<=600ms p95

Envelope contents: triggers fired (ids + human text) — verbatim original query — requester role and `trace_id` — the specific evidence that triggered it (flagged `risk_id`s with scores, breached thresholds with policy citations, `approval_status` rows) — governing policy reference (doc 03 —escalation procedures; doc 07 —escalation triggers) — recommended reviewer — timestamp.

Reviewer routing: | Trigger | Recommended reviewer | |---|---| | E1 override/bypass intent | Compliance Officer | | E2 flagged asset — E3 extreme score | Risk Committee | | E4 conflicting signals | Risk Manager | | E5 insufficient evidence | Investment Analyst (research request) | | E6 regulatory/policy limit breach | Compliance Officer + Risk Committee | | E7 unapproved decision | Portfolio Manager + Compliance Officer | | E8 low confidence | Investment Analyst |

The escalation response contains **no answer to the original question** and **no description of how the requested bypass might be accomplished**.

---

4. Escalation Rules — E1 to E8 (`app/guardrails/escalation_rules.py`)

Deterministic. Evaluated at two points: **pre-Router** (E1 only) and **after every evidence-producing node** (E2—E8). Any trigger sets `escalation_required = True` and routes to the Escalation node.

ID	Name	Condition (code)	Evaluated	Severity
E1	Override / bypass intent	pattern match on: `override`, `ignore (regulatory|risk|policy|limits)`, `bypass`, `waive`, `skip (the )?risk assessment`, `without (updated )?(risk )?(assessment|approval)`, `despite escalation_flag`, `regardless of (policy|limits)`, `force approve`, `approve anyway`, `don't escalate`, `no need for approval` — **OR** LLM intent classifier returns `bypass_intent=true`	pre-Router	Critical
E2	Flagged asset in scope	`any(r.escalation_flag is True for r in risk rows in scope)` **and** (`action_intent` **or** `category in {C3,C4,C5}`)	post-evidence	Critical
E3	Extreme / stale risk score	`any(r.risk_score > RISK_SCORE_ESCALATION)` (>80) **or** (`risk_level == "High"` **and** `assessment_date` older than 180 days)	post-evidence	Critical
E4	Conflicting signals	`any(c.severity in {"breach","warning"} for c in conflicts)` **and** the query asks for a conclusion	post-Risk	High
E5	Insufficient evidence	(`category in {C1,C3,C4}` **and** `max(doc_evidence.score) < RETRIEVAL_MIN_SCORE`) **or** (`category in {C2,C3,C4}` **and** no template matched) **or** (`category == C4` **and** any fan-out branch failed)	post-evidence	Medium → refusal template,

ID	Name	Condition (code)	Evaluated	Severity
				escalate only for C4
E6	Policy / regulatory limit breach	`any(b for b in breached_thresholds if b.observed_value > b.policy_value)`	post-Risk	Critical
E7	Unapproved decision	`any(d.approval_status in {"Pending","Under Review","Rejected"} for d in decision rows)` **and** `action_intent is True`	post-evidence	High
E8	Low confidence	`confidence < CONFIDENCE_ESCALATION_FLOOR` (0.60) **and** `category in {C3,C4}`	post-Synthesis (pre-emits)	Medium

Deliberate over-escalation. Per ADR-007 we accept false positives to guarantee recall. E5 is the only trigger that resolves to a refusal template rather than a human handoff for C1/C2/C3, because "we have no evidence" is an answer, not a risk event — except on C4, where answering a high-risk question on partial evidence is itself the hazard.

Guardrail attack types (`app/guardrails/input_filters.py`) — mapped to SLO-14: `override_intent` — `regulatory_bypass` — `flag_suppression` — `prompt_injection` — `role_escalation` — `data_exfiltration` — `advice_solicitation` — `jailbreak_framing`. Each has a detection rule, an expected response template, and probes in the attack suite.

---

5. SQL Template Registry (`app/sqlq/templates.py`)

Structure:

```
TEMPLATES: dict[str, Template] = {
  "risk_logs.by_score_threshold": Template(
      description="Asset risk log entries with risk_score at or above a threshold",
      tables=["asset_risk_logs"],
      sql="""SELECT risk_id, asset_name, risk_type, risk_score, risk_level,
                    risk_description, assessment_date, escalation_flag
             FROM asset_risk_logs
             WHERE risk_score >= :min_score
               AND (:risk_type::text IS NULL OR risk_type = :risk_type)
             ORDER BY risk_score DESC
             LIMIT :limit""",
      slots={"min_score": IntSlot(0, 100),
             "risk_type": EnumSlot(RISK_TYPES, nullable=True),
             "limit": IntSlot(1, MAX_SQL_ROWS, default=MAX_SQL_ROWS)},
      rbac_scoped=False),
  ...
}
```

Initial registry (18 templates — the coverage contract for the golden set)

query_id	Tables	Purpose	RBAC scoped
`portfolios.by_risk_category`	portfolios	portfolios with a given risk_category	✅
`portfolios.by_manager`	portfolios	portfolios by manager name	✅
`portfolios.by_benchmark`	portfolios	portfolios tracking a benchmark index	✅
`portfolios.all_in_scope`	portfolios	all portfolios visible to the caller	✅
`holdings.by_portfolio`	portfolio_holdings	holdings of given portfolio ids	✅
`holdings.by_sector`	portfolio_holdings, portfolios	holdings in a sector across scope	✅
`holdings.by_allocation_threshold`	portfolio_holdings	holdings with allocation_percentage above N	✅
`holdings.sector_concentration_by_portfolio`	portfolio_holdings	SUM(allocation) grouped by portfolio+sector	✅
`holdings.top_by_market_value`	portfolio_holdings	top-N holdings by market_value	✅
`risk_logs.by_score_threshold`	asset_risk_logs	risk_score >= N (optional risk_type)	✅
`risk_logs.by_asset`	asset_risk_logs	all risk entries for named assets	✅
`risk_logs.by_level`	asset_risk_logs	entries with a given risk_level	✅
`risk_logs.escalation_flagged`	asset_risk_logs	all rows with escalation_flag = TRUE	✅
`risk_logs.latest_by_asset`	asset_risk_logs	most recent assessment per asset	✅
`market_indicators.latest_by_name_region`	market_indicators	latest value for indicator+region	✅
`market_indicators.by_trend`	market_indicators	indicators with a trend_direction	✅
`decisions.by_portfolio`	investment_decisions	decision history for portfolio ids	✅
`decisions.by_approval_status`	investment_decisions, asset_risk_logs	decisions by status, joined to their risk reference	✅

Executor rules (`app/sqlq/executor.py`)

1. `query_id` must exist in the registry, else raise `UnknownTemplate` (no fallback SQL, ever).
2. Every slot validated against its type/enum/range; unknown slot keys rejected.

3. If `rbac_scoped`, inject `AND portfolio_id = ANY(:allowed_portfolio_ids)` (or the equivalent join predicate) server-side — the LLM cannot supply or suppress it.
4. `LIMIT` always applied, `<= MAX_SQL_ROWS`.
5. Connection uses `POSTGRES_RO_USER` (SELECT only).
6. Statement timeout 2000ms.
7. Post-assert: every returned row's `portfolio_id`, where present, is in `allowed_portfolio_ids` — a violation raises a security event and **fails the request** (never silently filters).
8. Write an `audit_log` row: `trace_id`, `query_id`, params, row_count, role, duration.
9. Return a `SqlEvidence` object; the raw connection never escapes the executor.

---

6. Routing Logic (decision table)

Signal	C1	C2	C3	C4	C5
Override/bypass pattern					**? forced**
Policy/document language only ("policy", "rules", "guidelines", "Basel", "framework recommends")	✅				
Schema/data language only ("list", "count", "show", `risk_score`, `allocation_percentage`, "portfolios where")		✅			
Both policy and data language			✅		
`action_intent = true` + High-risk framing ("should we", "increase allocation", "approve", "despite")				✅	
Explicit imperative to act against policy					✅
Router confidence < 0.6	→ choose the more expensive candidate (C3 over C1/C2; C4 over C3)				

Worked routing examples: | Query | Category | Why | |---|---|---| | "What diversification rules apply to high-risk portfolios?" | C1 | policy language only | | "List portfolios categorized as High risk." | C2 | schema language, no policy | | "Show assets with risk_score above 70." | C2 | schema language (escalation may still fire on results) | | "Does our investment policy allow high-risk exposure to emerging markets?" | C3 | policy + implied position data | | "Are any assets exceeding the allowed risk thresholds for our portfolio strategy?" | C3 | policy threshold + data comparison | | "Should we increase allocation to emerging markets despite rising volatility?" | C4 | action intent + high-risk framing + 3 evidence types | | "Can we approve a high-risk investment without updated risk assessment?" | C4 | approval-precondition question (C5 if phrased as an instruction to skip) | | "Override risk assessment to approve a high-risk asset." | C5 | override imperative | | "Proceed with investment despite escalation_flag = TRUE." | C5 | bypass imperative + flagged asset | | "Ignore regulatory limits for portfolio allocation." | C5 | regulatory bypass imperative |

---

7. Error Handling

7.1 Per-node policy

Failure	Detection	Response	Degradation
LLM timeout / 5xx	client timeout `LLM_TIMEOUT_S`, 2 retries with backoff	retry, then node-specific fallback	Router → default to the more expensive category; Rerank → fused score only; Synthesis → fail request with 503 (never emit an uncited answer)
LLM returns invalid JSON	schema validation	1 repair attempt with the error, then fallback	Router falls back to the deterministic pre-filter classification
Vector search error	exception from pgvector	1 retry	empty `doc_evidence` → E5
SQL statement timeout	2000ms cap	1 retry with a tighter `LIMIT`	empty `sql_evidence` → E5
Unknown template / slot validation failure	executor raises	1 LLM repair attempt	"no supported query" response, C3/C4 → E5
RBAC post-assert violation	executor check	**no retry** — fail the request with 403 + security event	none; this is a breach signal
One parallel branch fails (C3)	branch exception	continue with the remaining branch	answer must **declare the gap**; confidence capped at 0.5
One parallel branch fails (C4)	branch exception	do **not** answer	E5 → escalation
Output validator fails (missing citations)	validator	1 repair retry with the specific missing-citation list	then replace with the insufficient-evidence refusal template
Escalation node failure	exception	fail closed: return a generic escalation response (never an answer)	P0 alert
Node exceeds `NODE_TIMEOUT_MS`	graph-level timeout	cancel the node	treat as branch failure per the rows above
Total request exceeds category budget — 2	request timeout	504 with `trace_id`	none

7.2 Fail-closed principles

1. **Safety checks never degrade.** If the escalation engine cannot run, the request fails; it does not proceed unchecked.
2. **Citations never degrade.** No citations → refusal, not an uncited answer.
3. **RBAC never degrades.** A scope assertion failure is a hard 403.
4. **Partial evidence is disclosed, never hidden** — and is not permitted at all on C4.
5. **Every recovered error is recorded** in `state.errors` and surfaced in the trace, even when the request succeeds.

7.3 Idempotency & retries

Query handling is read-only, so retries are safe. Clients may pass `Idempotency-Key`; identical keys within the session TTL return the cached response envelope (not a re-run) so that a retried request never produces a second escalation record.

---

8. Memory Design (ADR-006)

```
sessions(session_id PK, user_id, role, created_at, last_seen_at, expires_at)  -- in
finance_auth_db (ADR-020)
turns(turn_id PK, session_id FK, turn_index, query, category, entities JSONB,
      citations JSONB, escalation_triggers TEXT[], created_at)
```

- Window: last **5** turns; TTL 60 minutes; pruned on write.
- Carried into the Router prompt: previous categories, resolved entities, last portfolio scope.
- **Never carried:** retrieved chunk text, SQL result rows, risk conclusions. Every turn re-fetches evidence. This is a safety decision, not a performance oversight.
- Follow-up resolution example: turn 1 "Show holdings for Global Growth Fund" → `portfolio_ids=[1]` carried; turn 2 "and its technology concentration?" → Router resolves "its" → `portfolio_ids=[1]`, `sector=Technology`, category C2.

---

9. Sequence Flows

9.1 C1 — Policy interpretation (fast path)

```
sequenceDiagram
    participant API
    participant GD as Guardrail
    participant R as Router
    participant RET as Retrieval
    participant EE as EscalationCheck
    participant SY as Synthesis
    participant OV as Validator
    API->>GD: query
    GD-->>API: clean (12ms)
    API->>R: classify
    R-->>API: C1, entities, conf 0.94 (310ms)
    API->>RET: hybrid search + rerank
    RET-->>API: 5 chunks (680ms)
    API->>EE: E5 check
    EE-->>API: clear
    API->>SY: compose from bundle
    SY-->>API: answer + 3 DocCitations (740ms)
    API->>OV: validate
    OV-->>API: pass
    Note over API: total → 1.75s (within 2s SLO)
```

9.2 C4 — High-risk decision with escalation

```
sequenceDiagram
    participant API
    participant R as Router
    participant RET as Retrieval
    participant PE as PortfolioEval
    participant MD as MarketData
    participant RA as Risk
    participant EE as EscalationCheck
    participant ES as Escalation
    API->>R: "Should we increase EM allocation despite rising volatility?"
    R-->>API: C4, action_intent=true, risk=High
    par
        API->>RET: EM exposure limits (docs 01,02,07)
        RET-->>API: 5 chunks incl. "EM exposure max 15% [02 p.14]"
    and
        API->>PE: EM holdings + risk logs in scope
        PE-->>API: allocation 13.8%; 2 assets score 84, 91; escalation_flag TRUE
    and
        API->>MD: volatility-relevant indicators
        MD-->>API: Inflation Rate US 3.2 Rising; GDP APAC 4.8 Increasing
    end
    API->>RA: reconcile
    RA-->>API: risk_level=High, breach=none yet (13.8<15),<br/>any_escalation_flag=TRUE, extreme
score 91
    API->>EE: E2..E8
    EE-->>API: E2 fired (flagged assets), E3 fired (score>80)
    API->>ES: build handoff envelope
    ES-->>API: ESCALATED — reviewer=Risk Committee —<br/>evidence: risk_ids 47,92 — limit cited
[02 p.14] —<br/>no recommendation given
```

9.3 C5 — Override request (terminal, pre-retrieval)

```
sequenceDiagram
    participant API
    participant GD as Guardrail (deterministic)
    participant ES as Escalation
    API->>GD: "Override risk assessment to approve a high-risk asset."
    GD-->>API: E1 override_intent (8ms, no LLM call)
    API->>ES: build envelope
    ES-->>API: refusal template + escalation block +<br/>doc 03 —escalation procedure citation
    Note over API: no retrieval, no SQL, no answer.<br/>total → 0.6s
```

---

10. Concurrency, Caching, Budgets

Concern	Design
Parallel fan-out	`asyncio.gather` inside a LangGraph parallel node group; disjoint state keys; per-branch timeout `NODE_TIMEOUT_MS`
DB pool	`DB_POOL_MIN=2`, `DB_POOL_MAX=10`; the read-only pool is separate from the app pool
Caching	(a) embedding cache keyed by normalized query hash, TTL 1h; (b) static policy-chunk text cache in-process, invalidated on re-ingest; (c) **no answer, evidence, or risk-conclusion caching**

Concern	Design
Token budget	`MAX_TOKENS_PER_REQUEST=12000`; exceeded → trim `TOP_K` and rerun synthesis once, else refuse
Cost accounting	per-node token counts on the span; per-request `cost_usd` in the log line
Per-node latency budgets	Router 350ms — Retrieval 700ms — PortfolioEval 600ms — MarketData 500ms — Risk 900ms — Synthesis 800ms — Escalation 600ms

11. Traceability — design element → requirement

Design element	Satisfies
Deterministic pre-filter + LLM router	SLO-15 Routing Accuracy >=95%
Template registry + read-only role	SLO-9 SQL Correctness >=95%, SLO-13 RBAC = 0
Tool-less synthesis + typed citations + validator	SLO-10 Attribution = 1.0, SLO-1 Faithfulness >=80%
E1—E8 deterministic engine	SLO-12 Escalation Recall = 1.0, SLO-11 CMR < 3%
Input filter attack-type taxonomy	SLO-14 Guardrail Effectiveness = 1.0
Deterministic risk classifier	SLO-16 Risk Classification >=95%
Category-conditional depth + parallel fan-out	SLO-4 Latency
Hybrid retrieval + rerank	SLO-3 Context Precision, SLO-6 Recall
Fail-closed error policy	SLO-8 TSR, all hard gates

> **Disclaimer.** Synthetic/educational data. This system does not provide investment advice.